
Regularization & Model Selection

BSAD 8310: Business Forecasting — Lecture 6

Department of Economics
University of Nebraska at Omaha
August 24, 2026

Lecture Outline

- 1 The Problem: Too Many Predictors
- 2 Shrinkage Methods
- 3 Choosing the Penalty
- 4 In Practice
- 5 Key Takeaways and Roadmap

The Problem: Too Many Predictors

Feature engineering gives us more predictors than we can safely estimate. Something has to decide which ones survive.

Why Regularize?

OLS minimizes $\sum_i (y_i - \mathbf{x}_i' \beta)^2$. A weekly model with lags, calendar dummies, promotions, and macro series can easily carry $k = 60$ predictors on $n = 150$ observations — OLS will fit that sample beautifully and forecast badly.

What goes wrong:

- Variance of $\hat{\beta}$ grows with k
- Correlated predictors give unstable, wildly large coefficients with opposing signs
- When $k > n$ there is no unique solution at all

Regularization: add a penalty on coefficient size,

$$\underbrace{\sum_i (y_i - \hat{y}_i)^2}_{\text{fit}} + \underbrace{\lambda \cdot \text{Penalty}(\beta)}_{\text{complexity}}$$

Accept a little bias to buy a large reduction in variance.

Subset Selection: The Classical Answer

Before shrinkage, the standard approach was to *count* predictors rather than shrink them (Guyon and Elisseeff 2003).

Best subset:

- Fit all 2^k models, pick the best
- Exact — hopeless past $k \approx 30$

Stepwise:

- Forward: add the best predictor, repeat
- Backward: start full, drop the weakest, repeat
- Greedy — can miss the best combination

This is the approach in *FPP* §7.5. Shrinkage replaces the in-or-out decision with a continuous dial — more stable, and easier to tune.

Chosen by: adjusted R^2 , AIC, AICc, BIC, or CV.

Subset selection is **discrete** — in or out. Small data changes flip predictors in and out, so the selected model is itself high-variance.

Shrinkage Methods

Keep every predictor, but pull its coefficient toward zero.

Ridge Regression (ℓ_2)

Ridge (Hoerl and Kennard 1970)

$$\hat{\beta}^{\text{ridge}} = \arg \min_{\beta} \sum_{i=1}^n (y_i - \mathbf{x}_i' \beta)^2 + \lambda \sum_{j=1}^k \beta_j^2$$

Behavior:

- Shrinks all coefficients, none exactly to zero
- $\lambda = 0$ is OLS; $\lambda \rightarrow \infty$ sends all $\beta_j \rightarrow 0$
- Handles correlated predictors gracefully — splits the coefficient between them

Use when you believe most predictors carry a little signal and you want to keep them all.

Standardize first. The penalty is not scale-invariant — dollars and thousands of dollars are penalized differently.

LASSO (Tibshirani 1996)

$$\hat{\beta}^{\text{lasso}} = \arg \min_{\beta} \sum_{i=1}^n (y_i - \mathbf{x}_i' \beta)^2 + \lambda \sum_{j=1}^k |\beta_j|$$

Behavior:

- Sets some coefficients **exactly** to zero
- Selection and estimation in one step
- Yields a sparse, readable model — valuable when you must explain the forecast

Use when you believe only a handful of predictors genuinely matter and you want the model to tell you which.

Among correlated predictors LASSO keeps *one* arbitrarily and zeros the rest. Do not read the survivor as “the important one.”

Why LASSO Zeros Out and Ridge Does Not

Both solve “minimize squared error subject to a budget on coefficient size.” The shape of the budget is what differs.

Ridge: budget $\sum_j \beta_j^2 \leq t$ — a **circle**.

- Smooth boundary with no corners
- The contours of the squared-error surface almost never touch it on an axis
- So coefficients get small, but not zero

LASSO: budget $\sum_j |\beta_j| \leq t$ — a **diamond**.

- Corners sit exactly on the axes
- Contours are very likely to first touch a corner
- Touching a corner means $\beta_j = 0$ for that predictor

The sparsity of LASSO is a geometric accident of the ℓ_1 ball having corners — not a statistical test of significance. A zeroed coefficient means “not worth its budget here,” not “proven irrelevant.” See James et al. (2023) §6.2 for the full picture.

Elastic Net: Both Penalties at Once

Elastic Net (Zou and Hastie 2005)

$$\sum_{i=1}^n (y_i - \mathbf{x}'_i \boldsymbol{\beta})^2 + \lambda \left[\alpha \sum_j |\beta_j| + (1 - \alpha) \sum_j \beta_j^2 \right]$$

λ sets the total penalty; $\alpha \in [0, 1]$ sets the mix ($\alpha = 1$ is LASSO, $\alpha = 0$ is Ridge).

Why the mix helps:

- Keeps LASSO's ability to zero out predictors
- Adds Ridge's **grouping effect**: correlated predictors enter or leave together rather than one arbitrarily surviving

Forecasting default. Lagged predictors are correlated almost by construction — y_{t-1} and y_{t-2} move together. Elastic Net is usually the safer starting point over pure LASSO.

Choosing the Penalty

λ is not estimated from the model. It is tuned — and how you tune it decides whether the model generalizes.

The Regularization Path

Fit the model across a grid of λ values and trace each coefficient.

Reading the path, left to right:

- At small λ : coefficients near their OLS values, model is dense and unstable
- As λ grows: coefficients shrink; under LASSO they drop to zero one by one
- At large λ : only the most robust predictors survive
- At $\lambda \rightarrow \infty$: intercept-only, the mean forecast

The **order** in which predictors leave is informative: the last survivors are the ones that hold up under the most pressure. Report this ordering — it is far more stable than a single model's coefficient table.

Choosing λ by Cross-Validation

Procedure:

1. Define a grid of λ , usually log-spaced
2. For each λ , run walk-forward CV and record mean validation RMSE
3. Pick the λ minimizing CV error
4. Refit on all training data at that λ

The one-standard-error rule: choose the largest λ whose CV error is within one standard error of the minimum — a simpler model at essentially the same accuracy.

Never use KFold here. Random folds put future observations in the training set and past ones in validation. The CV error will look excellent and the forecast will fail (Bergmeir et al. 2018).

Use `TimeSeriesSplit`, exactly as in Lecture 1's walk-forward evaluation.

In Practice

Standardize, tune on a time-aware split, and read the path.

Code Template

```
from sklearn.linear_model import ElasticNetCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import
TimeSeriesSplit

model = make_pipeline(
    StandardScaler(),
    ElasticNetCV(
        l1_ratio=[.1, .5, .7, .9, .95, 1],
        cv=TimeSeriesSplit(n_splits=5),
        random_state=42))

model.fit(X_train, y_train)
```

Notes:

- `make_pipeline` keeps scaling *inside* each CV fold — scaling on the full sample leaks information
- `l1_ratio` is α : the Ridge/LASSO mix
- ...CV variants tune λ ; plain Ridge/Lasso take it fixed
- `lasso_path()` returns the full path

`sklearn` calls the penalty `alpha`, not `lambda`.

Key Takeaways and Roadmap

Shrinkage is the linear-model counterpart to what trees do by pruning.

Linear Methods: Which One When

Method	Penalty	Zeros out?	Best when
OLS	none	no	$k \ll n$, predictors uncorrelated
Stepwise	discrete	yes	Few candidate predictors, need a story
Ridge	ℓ_2	no	Many predictors, all mildly useful
LASSO	ℓ_1	yes	Few predictors truly matter
Elastic Net	both	yes	Many <i>correlated</i> predictors

Connection to Lecture 5. Trees and shrinkage solve the same problem in different geometries. XGBoost's `reg_lambda` is literally a Ridge penalty on leaf weights, and its `gamma` penalizes leaf count the way LASSO penalizes nonzero coefficients. Regularization is not a linear-model topic — it is how every flexible model is kept honest.

Lecture 6: Key Takeaways

1. With many predictors OLS overfits: coefficient variance grows with k , and with $k > n$ there is no unique solution at all.
2. **Subset selection** counts predictors in or out. It is discrete, greedy, and unstable — small data changes flip the selected model.
3. **Shrinkage** replaces that switch with a dial. Ridge (ℓ_2) shrinks everything smoothly; LASSO (ℓ_1) shrinks and zeros out; Elastic Net does both and keeps correlated predictors together.
4. **Standardize first**, inside each CV fold — none of these penalties are scale-invariant. λ **is tuned, not estimated**: use `TimeSeriesSplit`, never random K -fold, which leaks the future.
5. The **regularization path** — the order in which predictors drop out — is a more stable finding than any single coefficient table.

Next: Introduction to Neural Networks (PyTorch) — where these same penalties reappear as weight decay and dropout.

References I

-  Bergmeir, Christoph, Rob J. Hyndman, and Bonsoo Koo (2018). “A Note on the Validity of Cross-Validation for Evaluating Autoregressive Time Series Prediction”. In: *Computational Statistics & Data Analysis* 120, pp. 70–83.
-  Guyon, Isabelle and André Elisseeff (2003). “An Introduction to Variable and Feature Selection”. In: *Journal of Machine Learning Research* 3, pp. 1157–1182.
-  Hoerl, Arthur E. and Robert W. Kennard (1970). “Ridge Regression: Biased Estimation for Nonorthogonal Problems”. In: *Technometrics* 12.1, pp. 55–67.
-  James, Gareth et al. (2023). *An Introduction to Statistical Learning with Applications in Python*. 1st. New York: Springer. URL: <https://www.statlearning.com/>.
-  Tibshirani, Robert (1996). “Regression Shrinkage and Selection via the Lasso”. In: *Journal of the Royal Statistical Society: Series B* 58.1, pp. 267–288.
-  Zou, Hui and Trevor Hastie (2005). “Regularization and Variable Selection via the Elastic Net”. In: *Journal of the Royal Statistical Society: Series B* 67.2, pp. 301–320.